

Relazione Prova Finale di Reti Logiche

Francesco Lo Conte

Codice Persona: 10851673 Matricola: 216370

Antonio Gabriele La Vecchia

Codice Persona: 10847526 Matricola: 213239

POLITECNICO
MILANO 1863

CONTENTS

1	Introduzione	3
1.1	Obiettivi del progetto	3
1.2	Specifica del funzionamento	3
1.3	Specifica Hardware.....	4
2	Architettura	5
2.1	Registro dei coefficienti.....	6
2.1.1	Interfaccia	6
2.1.2	Architettura	7
2.2	Shift Register	9
2.2.1	Interfaccia	9
2.2.2	Architettura	10
2.3	Filter.....	11
2.3.1	Interfaccia	11
2.3.2	Architettura	12
2.4	Normalizer	13
2.4.1	Interfaccia	13
2.4.2	Architettura	14
2.5	Address Manager	16
2.5.1	Interfaccia	16
2.5.2	Architettura	17
2.6	FSM	19
2.6.1	Interfaccia	19
2.6.2	Architettura	20
3	Risultati Sperimentali	24
3.1	Sintesi	24
3.2	Simulazioni	24
3.2.1	Testbench fornito dal docente.....	24
3.2.2	Test di saturazione	24
3.2.3	Test con filtro di ordine 5	26
3.2.4	Test di robustezza	26
3.2.5	Elaborazioni consecutive senza reset	26
3.2.6	Reset durante l'elaborazione	26
3.2.7	Test con lunghezza minima	27
4	Conclusioni	28

1 Introduzione

1.1 Obiettivi del progetto

Il progetto consiste nello sviluppare un componente hardware nel linguaggio VHDL che ha il compito di leggere una sequenza di parole numeriche (W) da una memoria, applicare a questa sequenza un filtro differenziale per calcolare una nuova sequenza di risultati (R), e infine salvare quest'ultima in memoria. La configurazione del filtro (ordine, coefficienti) e la lunghezza della sequenza sono anch'esse memorizzate in memoria e possono variare ad ogni esecuzione.

1.2 Specifica del funzionamento

Ogni parola della sequenza, sia in ingresso (W) che in uscita (R), può assumere un valore compreso tra -128 e +127 ed è rappresentata in memoria come un byte in complemento a due.

L'intera elaborazione è guidata da un preambolo di 17 byte che precede la sequenza W e che contiene i parametri operativi:

- la lunghezza K della sequenza (data dalla concatenazione dei primi due byte, K1 e K2), per una lunghezza minima di 7 byte;
- un selettore S, che indica quale filtro utilizzare, in base al bit meno significativo del parametro stesso ('0' per filtro di ordine 3, '1' per ordine 5);
- i 14 coefficienti C, 7 per ogni filtro (i primi 7 per il filtro di ordine 3, gli ultimi per ordine 5).

Il componente legge questi parametri e applica la seguente logica:

- Il calcolo del filtro per ogni elemento i della sequenza è definito dalla formula:
$$f(i) = (1/n) \cdot \sum C_j \cdot f[j+i]$$
, con j che va da -1 a +1; e con l che vale 2 per il filtro di ordine 3, e 3 per il filtro di ordine 5. n è il fattore di normalizzazione, che vale 12 per il filtro di ordine 3, 60 per il filtro di ordine 5. I coefficienti C_j sono quelli letti dalla memoria.
- Per i calcoli ai bordi della sequenza (all'inizio e alla fine), le parole che cadono al di fuori del vettore W vengono considerate pari a 0.
- Il risultato di ogni calcolo viene normalizzato tramite divisioni approssimate con shift aritmetici a destra. Per ridurre l'errore sui numeri negativi, viene sommato +1 al risultato di ogni shift se il valore di partenza è negativo.
- Il risultato finale normalizzato viene saturato all'estremo più vicino per rientrare nell'intervallo [-128, +127], qualora ne fosse al di fuori, prima di essere scritto in memoria.
- La sequenza dei risultati R viene scritta in memoria a partire dall'indirizzo successivo all'ultima parola W, ovvero all'indirizzo $ADD + 17 + K$.

In seguito, un esempio con filtro di ordine 3 (valori in decimale):

Sequenza di partenza (W):

[32 -24 -35 0 46 -54 -39 -22 -53 -47 12 11 11 45 -30 -14 -35 -25 -19 -35 -41 -61 -24 -62]

Sequenza finale (R):

[11 43 -13 -54 33 53 -28 8 18 -38 -31 7 -24 23 33 -1 7 -11 5 10 15 -12 3 -10]

1.3 Specifica Hardware

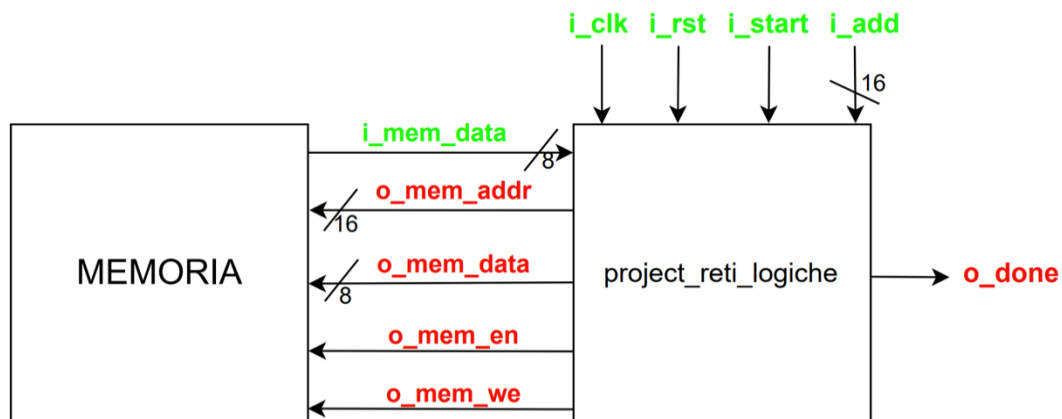


Figure 1: Rappresentazione schematica dell'interfaccia del componente **project_reti_logiche**.

Ingressi

- **i_clk**: segnale di clock;
- **i_rst**: segnale di reset asincrono;
- **i_add**: vettore da 16 bit, indica l'indirizzo di memoria a partire dal quale inizia il preambolo di configurazione;
- **i_start**: segnale di inizio della computazione;
- **i_mem_data**: vettore da 8 bit, contiene il dato letto dalla memoria in seguito a una richiesta di lettura.

Uscite

- **o_done**: segnale che va alto quando l'elaborazione è stata completata e rimane tale finché **i_start** non torna basso;
- **o_mem_en**: segnale che abilita la comunicazione con la memoria (sia in lettura che in scrittura);
- **o_mem_we**: segnale che abilita la scrittura in memoria. Se '1' si esegue una scrittura, se '0' una lettura;
- **o_mem_addr**: vettore da 16 bit, indica l'indirizzo di memoria su cui scrivere o leggere;
- **o_mem_data**: vettore da 8 bit, indica il valore da scrivere in memoria.

Al componente verrà sempre dato un segnale di reset prima della prima elaborazione, ma dovrà essere in grado di gestire elaborazioni multiple successive senza la necessità di un reset intermedio.

2 Architettura

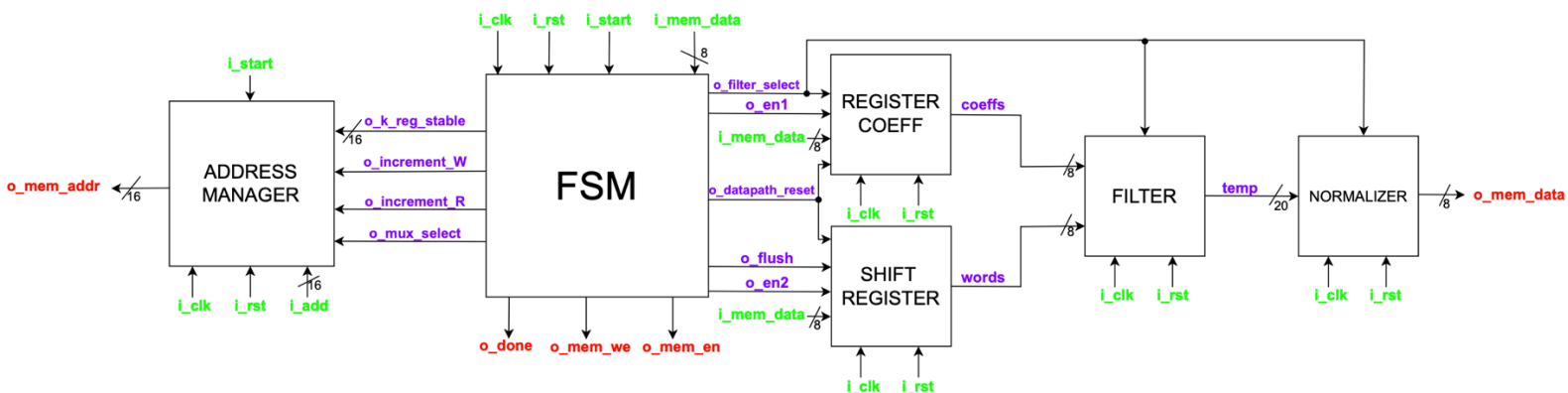


Figure 2: Struttura della rete.

In **rosso** sono rappresentate le **uscite principali** del componente.

In **verde** sono rappresentate le **entrate principali** del componente.

In **viola** sono rappresentati i **segnali** dei moduli.

Il sistema è composto da 6 componenti principali, in seguito analizzati approfonditamente:

- **FSM:** Macchina a stati finiti che agisce come controllore centrale. Si occupa di orchestrare l'intero processo, dalla lettura dei parametri iniziali alla scrittura dei risultati finali, generando i segnali di controllo per tutti gli altri moduli.
- **Address_Manager:** Unità dedicata esclusivamente alla gestione degli indirizzi di memoria. Mantiene al suo interno due puntatori separati per la lettura (**w_addr_ptr**) e la scrittura (**r_addr_ptr**), e fornisce alla memoria l'indirizzo corretto in base ai comandi ricevuti dalla FSM.
- **Register_coeff:** Un registro "intelligente" che si occupa di caricare i coefficienti del filtro. In base al segnale **i_filter_select**, ascolta i 14 coefficienti letti dalla memoria ma salva al suo interno solo i 7 pertinenti all'elaborazione richiesta, ignorando gli altri.
- **Shift_register:** Un registro a scorrimento che implementa la "finestra mobile" di 7 parole necessaria al calcolo. Durante la fase di elaborazione, carica le parole dalla memoria (**i_mem_data**) e, durante la fase finale di svuotamento, carica zeri per permettere il calcolo degli ultimi risultati.
- **Filter:** È il motore matematico del progetto. Ad ogni ciclo di clock, riceve i 7 coefficienti da **Register_coeff** e la finestra di 7 parole dallo **Shift_register** ed esegue il calcolo della sommatoria pesata, producendo un risultato intermedio a 20 bit.
- **Normalizer:** Rappresenta lo stadio finale del datapath. Prende il risultato a 20 bit dal **Filter**, lo normalizza tramite shift aritmetici (applicando la correzione per i numeri negativi) e lo satura nell'intervallo $[-128, +127]$ per produrre il dato finale a 8 bit da scrivere in memoria.

2.1 Registro dei coefficienti

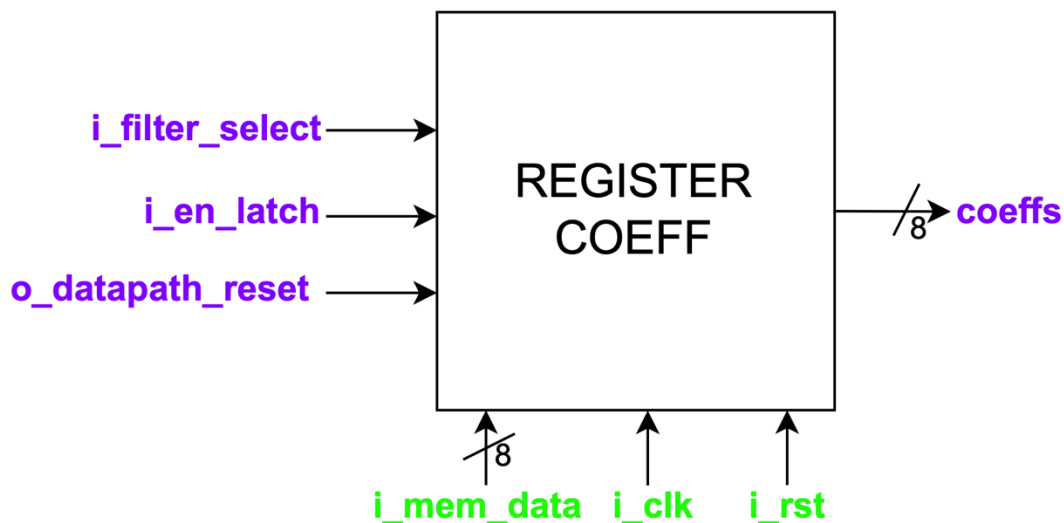


Figure 3: Rappresentazione schematica del **Register_coeff**.

Il **Register_coeff** (registro dei coefficienti) è un componente sincrono il cui scopo è leggere e memorizzare i coefficienti del filtro. Poiché i coefficienti sono letti sequenzialmente dalla memoria, questo modulo li cattura uno alla volta e, in base al filtro selezionato, salva i 7 coefficienti pertinenti in un registro interno.

2.1.1 Interfaccia

- **i_clk**: Segnale di clock per la sincronizzazione del componente.
- **i_rst**: Segnale di reset asincrono, che azzerà lo stato interno del componente, inclusi i registri dei coefficienti e i contatori. Questo ingresso è collegato all'OR logico tra il reset globale del sistema (**i_rst**) e il segnale **o_datapath_reset** generato dalla **FSM**. Questo garantisce che il componente venga azzerato sia all'accensione sia all'inizio di ogni nuova elaborazione, permettendo le esecuzioni consecutive.
- **i_mem_data**: Vettore a 8 bit che trasporta il singolo byte di coefficiente letto dalla memoria.
- **i_en_latch**: Segnale di abilitazione proveniente dalla **FSM**. Quando è alto, il componente cattura il valore su **i_mem_data**.
- **i_filter_select**: Segnale dalla **FSM** che indica quale filtro è stato scelto. Se vale '0' (filtro di ordine 3) si catturano i primi 7 coefficienti, altrimenti se vale '1' (filtro di ordine 5), si catturano gli ultimi 7.
- **coeffs**: Vettore di output di tipo **byte_array** che contiene i 7 coefficienti del filtro selezionato, pronti per essere usati dal componente **Filter**.

2.1.2 Architettura

```
architecture Behavioral of Register_coeff is
    signal coeff_regs    : byte_array := (others => (others => '0'));
    signal read_counter  : natural range 0 to 14 := 0;
    signal write_counter  : natural range 0 to 7 := 0;
    signal last_en       : std_logic := '0';

begin

    process(i_clk, i_rst)
    begin
        if i_rst = '1' then
            coeff_regs <= (others => (others => '0'));
            read_counter <= 0;
            write_counter <= 0;
            last_en <= '0';
        elsif rising_edge(i_clk) then
            last_en <= i_en_latch;
            if i_en_latch = '1' and last_en = '0' then
                read_counter <= 0;
                write_counter <= 0;
            end if;
            if i_en_latch = '1' then
                if i_filter_select = '0' and read_counter < 7 then
                    coeff_regs(write_counter) <= i_mem_data;
                    write_counter <= write_counter + 1;
                elsif i_filter_select = '1' and read_counter >= 7 and read_counter < 14 then
                    coeff_regs(write_counter) <= i_mem_data;
                    write_counter <= write_counter + 1;
                end if;
                if read_counter < 14 then
                    read_counter <= read_counter + 1;
                end if;
            end if;
        end if;
    end process;

    coeffs <= coeff_regs;

end architecture Behavioral;
```

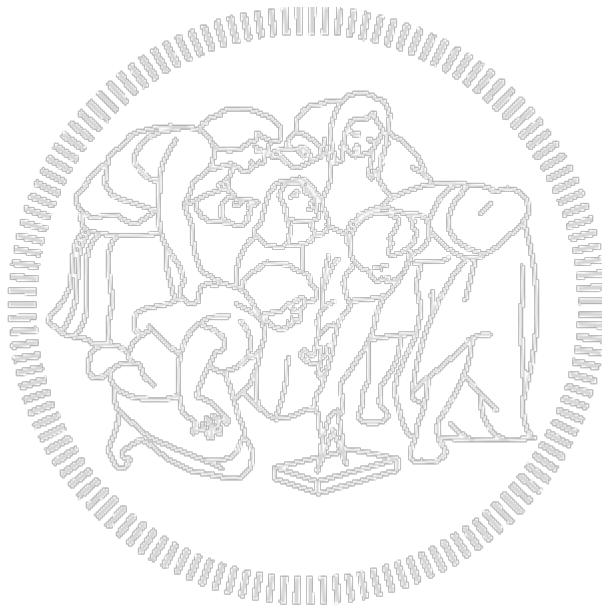
L'architettura è implementata come un processo sincrono, sensibile al fronte di salita del clock e al reset asincrono. All'interno sono presenti un registro a 7 posizioni per i coefficienti e due contatori per gestire la logica di lettura e scrittura.

Il funzionamento è il seguente:

- Un contatore interno, **read_counter**, tiene traccia di quale dei 14 coefficienti totali si sta leggendo dalla memoria.

- Un secondo contatore, **write_counter**, indica la posizione (da 0 a 6) in cui salvare il coefficiente nell'array interno **coeff_regs**.
- Quando la FSM alza il segnale **i_en_latch**, ad ogni ciclo di clock il **read_counter** avanza.
- Il valore di **i_filter_select** determina quali coefficienti salvare:
 - Se **i_filter_select** è '0' (filtro di ordine 3), vengono salvati in **coeff_regs** solo i primi 7 coefficienti letti (quando **read_counter** è tra 0 e 6).
 - Se **i_filter_select** è '1' (filtro di ordine 5), vengono ignorati i primi 7 coefficienti e vengono salvati in **coeff_regs** i 7 successivi (quando **read_counter** è tra 7 e 13).

Una volta memorizzati, i 7 coefficienti utili sono costantemente disponibili sull'uscita **coeffs** per essere utilizzati dal blocco **Filter**.



2.2 Shift Register

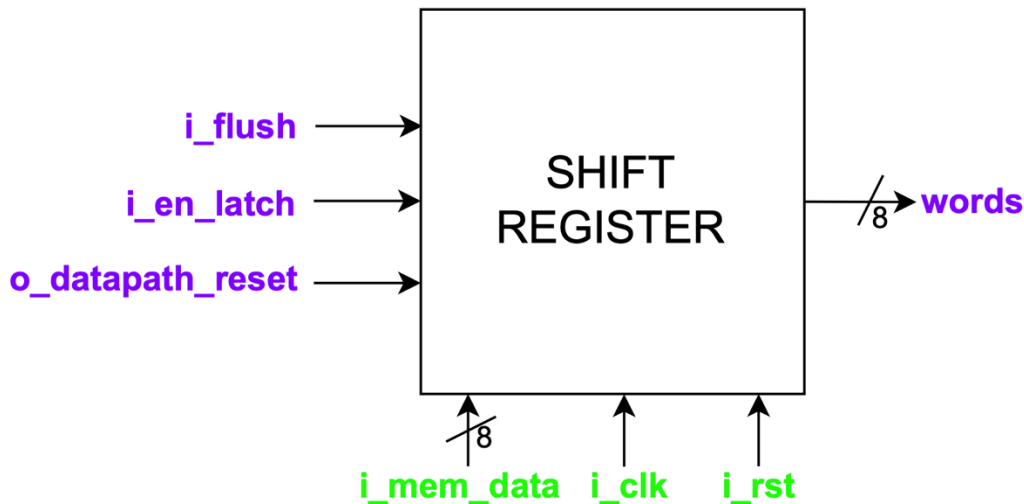


Figure 4: Rappresentazione schematica dello **Shift_register**.

Lo **Shift_register** è un registro a scorrimento progettato per mantenere una copia delle ultime 7 parole lette dalla memoria. La sua funzione è quella di fornire al blocco **Filter** una "finestra mobile" sui dati, consentendogli l'accesso simultaneo a più parole consecutive della sequenza W, necessarie per il calcolo della sommatoria pesata.

2.2.1 Interfaccia

- **i_clk**: Segnale di clock per la sincronizzazione delle operazioni.
- **i_rst**: Segnale di reset asincrono per inizializzare il contenuto del registro a zero. Come per il **Register_coeff**, questo ingresso nel design finale è collegato all'OR logico tra il reset globale del sistema (**i_rst**) e il segnale **o_datapath_reset** generato dalla **FSM**. Questo garantisce che la finestra di dati venga svuotata sia all'accensione sia all'inizio di ogni nuova elaborazione.
- **i_en_latch**: Segnale di abilitazione proveniente dalla **FSM**. Quando è attivo ('1'), il registro acquisisce una nuova parola da **i_mem_data**.
- **i_flush**: Segnale di "pulizia" proveniente dalla **FSM**. Quando è attivo, il registro carica un valore nullo (zero) per gestire le condizioni al contorno a fine sequenza.
- **i_mem_data**: Vettore a 8 bit che rappresenta la parola letta dalla memoria, pronta per essere inserita nel registro.
- **words**: Vettore di output di tipo **byte_array** che contiene la finestra delle 7 parole più recenti, inviata in input al componente **Filter**.

2.2.2 Architettura

```
architecture Behavioral of Shift_register is
    signal temp : byte_array := (others => (others => '0'));
begin
    process(i_clk, i_rst)
    begin
        if i_rst = '1' then
            temp <= (others => (others => '0'));
        elsif rising_edge(i_clk) then
            if i_en_latch = '1' or i_flush = '1' then
                temp(0 to 5) <= temp(1 to 6);
            end if;
            if i_en_latch = '1' then
                temp(6) <= i_mem_data;
            elsif i_flush = '1' then
                temp(6) <= (others => '0');
            end if;
        end if;
    end process;

    words <= temp;

end architecture Behavioral;
```

Il componente è implementato come un registro a scorrimento sincrono di 7 posizioni, ciascuna contenente un byte. La sua logica è gestita da un singolo processo sensibile al fronte di salita del clock e al segnale di reset asincrono.

Il suo comportamento è il seguente:

- Ad ogni fronte di salita del clock, se il segnale **i_en_latch** o **i_flush** è attivo, i dati all'interno del registro vengono fatti scorrere di una posizione (l'elemento in posizione *i* va in posizione *i-1*).
- Contemporaneamente, la prima posizione del registro (quella da cui "entrano" i nuovi dati) viene aggiornata:
 - Se **i_en_latch** è '1', viene caricata la nuova parola proveniente da **i_mem_data**. Questo è il funzionamento standard durante la lettura della sequenza W.
 - Se **i_flush** è '1', viene caricato un vettore di zeri. Questa operazione è essenziale alla fine del processo di filtraggio per "svuotare" la pipeline del filtro con valori nulli, permettendo così il calcolo corretto degli ultimi elementi della sequenza R secondo le condizioni al contorno della specifica.

Il contenuto della finestra di dati memorizzata nel registro interno **temp** è costantemente disponibile sul segnale di uscita **words**.

2.3 Filter

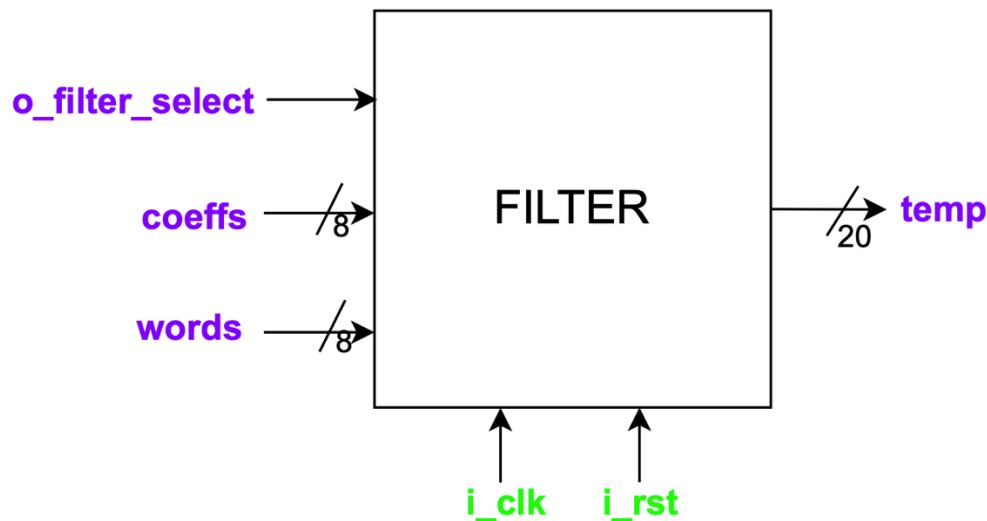


Figure 5: Rappresentazione schematica del **Filter**.

Il componente **Filter** ha il compito di eseguire l'operazione di filtraggio vera e propria. Ad ogni ciclo di clock, riceve la finestra di dati dallo **Shift_register** e i coefficienti dal **Register_coeff**, per poi applicare la formula matematica come definito dalla specifica.

2.3.1 Interfaccia

- **i_clk**: Segnale di clock per la sincronizzazione.
- **i_rst**: Segnale di reset asincrono per azzerare l'uscita.
- **o_filter_select**: Segnale in ingresso dalla **FSM** che determina quale set di coefficienti e quale logica di calcolo utilizzare ('0' per ordine 3, '1' per ordine 5).
- **coeffs**: Vettore di tipo **byte_array** in ingresso, contenente i 7 coefficienti forniti dal **Register_coeff**.
- **words**: Vettore di tipo **byte_array** in ingresso, che rappresenta la finestra dei 7 dati più recenti provenienti dallo **Shift_register**.
- **temp**: Segnale di uscita a 20 bit. Emette il risultato della sommatoria pesata, prima che questo venga normalizzato. La dimensione a 20 bit è stata scelta per garantire che non si verifichino overflow durante le moltiplicazioni e le somme intermedie.

2.3.2 Architettura

```
architecture filter_arch of filter is
    signal result : signed(19 downto 0);
begin
    process(i_clk, i_rst)
        variable temp_sum : signed(19 downto 0);
        variable mult_res : signed(15 downto 0);
    begin
        if i_rst = '1' then
            result <= (others => '0');
        elsif rising_edge(i_clk) then
            temp_sum := (others => '0');

            if o_filter_select = '0' then
                for i in 1 to 5 loop
                    mult_res := signed(coeffs(i)) * signed(words(i));
                    temp_sum := temp_sum + mult_res;
                end loop;
            else
                for i in 0 to 6 loop
                    mult_res := signed(coeffs(i)) * signed(words(i));
                    temp_sum := temp_sum + mult_res;
                end loop;
            end if;

            result <= temp_sum;
        end if;
    end process;

    temp <= std_logic_vector(result);
end architecture filter_arch;
```

L'architettura del filtro è realizzata tramite un processo sincrono che, ad ogni fronte di salita del clock, esegue il calcolo del valore filtrato. La logica è la seguente:

- Una variabile interna **temp_sum** viene inizializzata a zero ad ogni ciclo.
- Il segnale **o_filter_select** viene controllato per decidere quale operazione eseguire:
 - Se è **'0'** (filtro di ordine 3), un ciclo for itera da 1 a 5, moltiplicando tra loro gli elementi corrispondenti degli array **coeffs** e **words** (**coeffs(i) * words(i)**) e accumulando il risultato in **temp_sum**.
 - Se è **'1'** (filtro di ordine 5), un ciclo for simile itera da 0 a 6, utilizzando quindi tutti e 7 i coefficienti e le parole disponibili.
- Il risultato finale della sommatoria viene salvato nel segnale interno a 20 bit **result** e quindi reso disponibile sull'uscita **temp** per essere processato dal componente **Normalizer**.

2.4 Normalizer

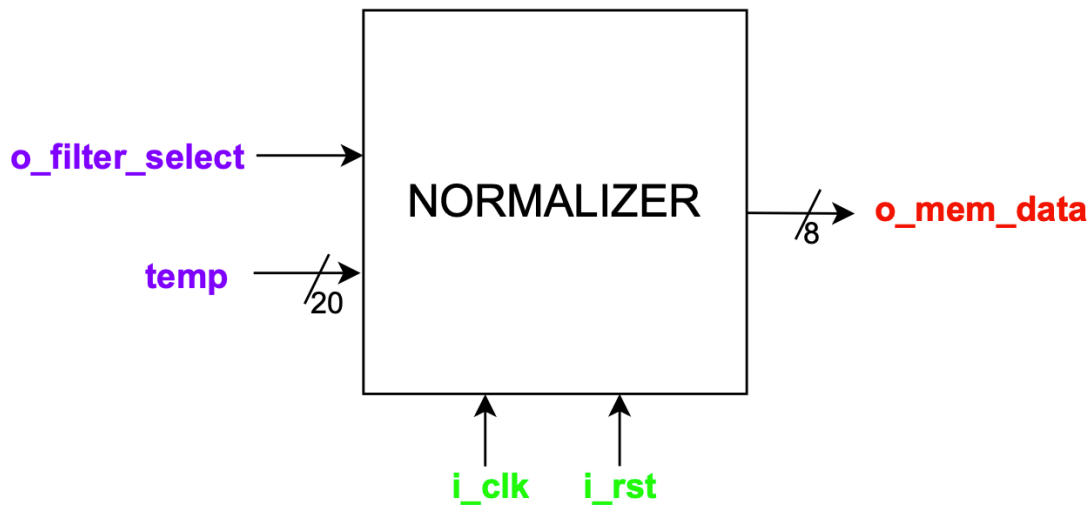


Figure 6: Rappresentazione schematica del **Normalizer**.

Il **Normalizer** è l'ultimo stadio del datapath di elaborazione. Questo componente sincrono riceve il risultato grezzo a 20 bit dal **Filter** e si occupa di due compiti critici: eseguire la divisione per il fattore di normalizzazione e saturare il risultato finale per rappresentarlo correttamente su 8 bit.

2.4.1 Interfaccia

- **i_clk**: Segnale di clock per la sincronizzazione.
- **i_rst**: Segnale di reset asincrono.
- **o_filter_select**: Segnale in ingresso dalla **FSM**, utilizzato per selezionare la corretta formula di normalizzazione ($n=12$ o $n=60$).
- **temp**: Vettore a 20 bit in ingresso, che rappresenta il risultato non normalizzato proveniente dal **Filter**.
- **o_mem_data**: Vettore di uscita a 8 bit, che rappresenta il valore finale del dato filtrato, normalizzato e saturato, pronto per la scrittura in memoria.

2.4.2 Architettura

```
architecture Normalizer_arch of Normalizer is
begin
  process(i_clk, i_rst)
    variable temp_val : signed(19 downto 0);
    variable x1, x2, x3, x4: signed(19 downto 0);
    variable sum_val : signed(19 downto 0);
  begin
    if i_rst = '1' then
      o_mem_data <= (others => '0');
    elsif rising_edge(i_clk) then
      temp_val := signed(temp);

      if o_filter_select = '0' then
        x1 := shift_right(temp_val, 4);
        x2 := shift_right(temp_val, 6);
        x3 := shift_right(temp_val, 8);
        x4 := shift_right(temp_val, 10);
        if temp_val < 0 then
          x1 := x1 + 1; x2 := x2 + 1; x3 := x3 + 1; x4 := x4 + 1;
        end if;
        sum_val := x1 + x2 + x3 + x4;
      else
        x1 := shift_right(temp_val, 6); x2 := shift_right(temp_val, 10);
        if temp_val < 0 then
          x1 := x1 + 1; x2 := x2 + 1;
        end if;
        sum_val := x1 + x2;
      end if;
      if sum_val > 127 then
        o_mem_data <= std_logic_vector(to_signed(127, 8));
      elsif sum_val < -128 then
        o_mem_data <= std_logic_vector(to_signed(-128, 8));
      else
        o_mem_data <= std_logic_vector(resize(sum_val, 8));
      end if;
    end if;
  end process;
end architecture Normalizer_arch;
```

Il componente implementa la sua logica all'interno di un singolo processo, eseguendo due operazioni in cascata ad ogni fronte di salita del clock: la normalizzazione e la saturazione.

1. Normalizzazione:

- Il componente legge il segnale **o_filter_select** per decidere come approssimare la divisione.
- Se il filtro selezionato è di ordine 3 (n=12), il valore **temp** viene shiftato a destra di 4, 6, 8 e 10 posizioni; i quattro risultati vengono poi sommati.

- Se il filtro selezionato è di ordine 5 ($n=60$), il valore **temp** viene shiftato a destra di 6 e 10 posizioni e i due risultati vengono sommati.
- Per ridurre l'errore introdotto dal troncamento sui numeri negativi, come da specifica, viene applicata una correzione: se il valore **temp** in ingresso è negativo, al risultato di ogni singolo shift viene sommato 1 prima di eseguire la somma finale.

2. Saturazione:

- Il valore normalizzato (**sum_val**) viene confrontato con i limiti del range rappresentabile su 8 bit in complemento a due.
- Se il valore è maggiore di 127, l'uscita viene saturata a 127.
- Se il valore è minore di -128, l'uscita viene saturata a -128.
- Se il valore rientra nel range, viene semplicemente ridimensionato a 8 bit.

Il valore finale, normalizzato e saturato, viene quindi posto sul segnale di uscita **o_mem_data**.



POLITECNICO
MILANO 1863

2.5 Address Manager

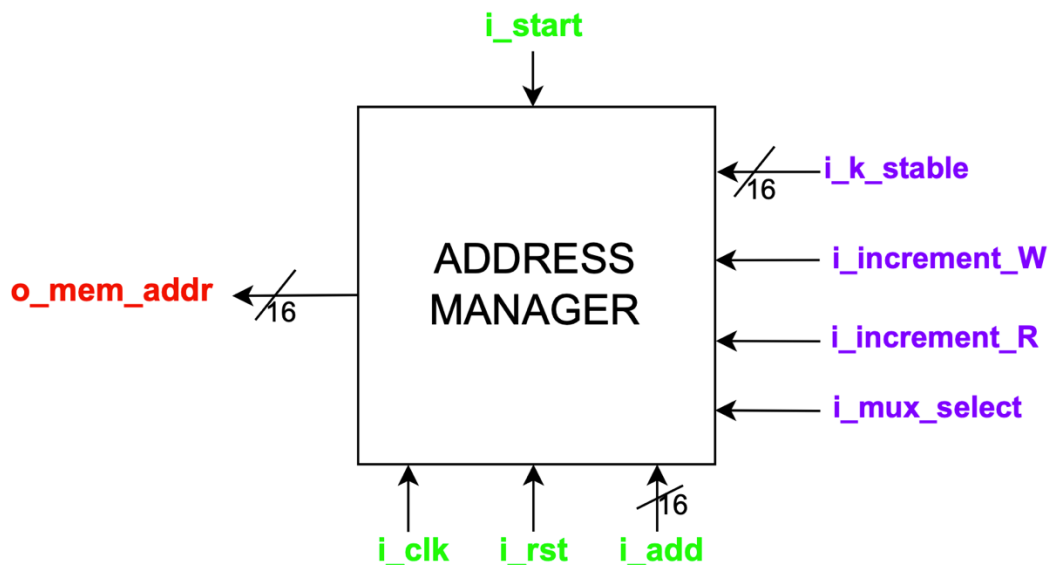


Figure 7: Rappresentazione schematica dell'Address_Manager.

L' **Address_Manager** ha il compito di generare e gestire tutti gli indirizzi di memoria richiesti durante l'esecuzione. Mantiene due puntatori distinti: uno per la lettura dei parametri e della sequenza di ingresso W, e uno per la scrittura della sequenza di uscita R. Un multiplexer interno seleziona quale dei due puntatori inviare alla memoria, in base alle direttive della **FSM**.

2.5.1 Interfaccia

- **i_clk**: Segnale di clock per la sincronizzazione.
- **i_rst**: Segnale di reset asincrono per azzerare i puntatori interni.
- **i_add**: Vettore a 16 bit che corrisponde all'indirizzo di base della memoria.
- **i_start**: Segnale di start che indica l'inizio di una nuova elaborazione e agisce come "soft reset" per la reinizializzazione dei puntatori.
- **i_k_stable**: Vettore a 16 bit che contiene il valore della lunghezza K della sequenza, fornito dalla **FSM** dopo averlo letto dalla memoria.
- **i_increment_w**: Segnale di controllo dalla **FSM** che comanda l'incremento del puntatore di lettura.
- **i_increment_r**: Segnale di controllo dalla **FSM** che comanda l'incremento del puntatore di scrittura.
- **i_mux_select**: Segnale di selezione dalla **FSM** che sceglie quale indirizzo (lettura o scrittura) fornire in uscita.
- **o_mem_addr**: Vettore di uscita a 16 bit che rappresenta l'indirizzo effettivo da inviare alla memoria.

2.5.2 Architettura

```
architecture Behavioral of Address_Manager is
    signal w_addr_ptr : unsigned(15 downto 0) := (others => '0');
    signal r_addr_ptr : unsigned(15 downto 0) := (others => '0');
    signal init_r_done : std_logic := '0';
    signal last_start : std_logic := '0';
begin
    process(i_clk, i_rst)
    begin
        if i_rst = '1' then
            w_addr_ptr <= (others => '0');
            r_addr_ptr <= (others => '0');
            init_r_done <= '0';
            last_start <= '0';
        elsif rising_edge(i_clk) then

            last_start <= i_start;
            if (i_start = '1' and last_start = '0') then
                w_addr_ptr <= unsigned(i_add);
            elsif i_increment_w = '1' then
                w_addr_ptr <= w_addr_ptr + 1;
            end if;
            if (i_start = '1' and last_start = '0') then
                init_r_done <= '0';
            end if;

            if init_r_done = '0' and i_k_stable /= X"0000" then
                r_addr_ptr <= unsigned(i_add) + 17 + unsigned(i_k_stable);
                init_r_done <= '1';
            elsif i_increment_r = '1' then
                r_addr_ptr <= r_addr_ptr + 1;
            end if;

        end if;
    end process;

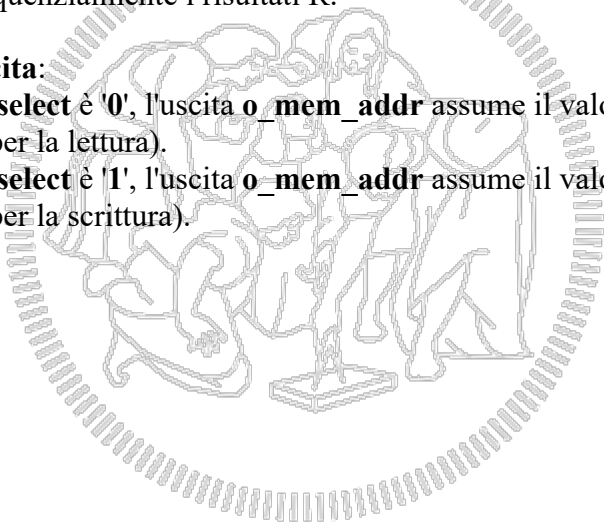
    o_mem_addr <= std_logic_vector(r_addr_ptr) when i_mux_select = '1' else
        std_logic_vector(w_addr_ptr);

end architecture Behavioral;
```

L'architettura è basata su un processo sincrono che gestisce lo stato e l'aggiornamento di due puntatori. La logica è progettata per gestire correttamente sia il reset hardware iniziale sia la reinizializzazione tra elaborazioni consecutive.

Il suo funzionamento si articola in tre fasi:

1. **Inizializzazione:** al reset (**i_rst**), i puntatori sono azzerati. All'inizio di una nuova elaborazione (rilevata dal fronte di salita di **i_start**), il puntatore di lettura **w_addr_ptr** viene immediatamente caricato con il valore di **i_add**. Il puntatore di scrittura **r_addr_ptr** viene inizializzato solo dopo che il valore K è stato letto e reso disponibile dalla FSM su **i_k_stable**. In quel momento, viene calcolato l'indirizzo di destinazione dei risultati come $i_add + 17 + K$. Un flag interno, **init_r_done**, assicura che questa operazione avvenga una sola volta per elaborazione.
2. **Incremento:** Durante l'esecuzione, i puntatori vengono incrementati in base ai segnali di controllo della FSM.
 - **w_addr_ptr** si incrementa di uno quando **i_increment_w** è alto, permettendo di leggere sequenzialmente i parametri e i dati W.
 - **r_addr_ptr** si incrementa di uno quando **i_increment_r** è alto, permettendo di scrivere sequenzialmente i risultati R.
3. **Selezione dell'Uscita:**
 - Se **i_mux_select** è '0', l'uscita **o_mem_addr** assume il valore di **w_addr_ptr** (indirizzo per la lettura).
 - Se **i_mux_select** è '1', l'uscita **o_mem_addr** assume il valore di **r_addr_ptr** (indirizzo per la scrittura).



POLITECNICO
MILANO 1863

2.6 FSM

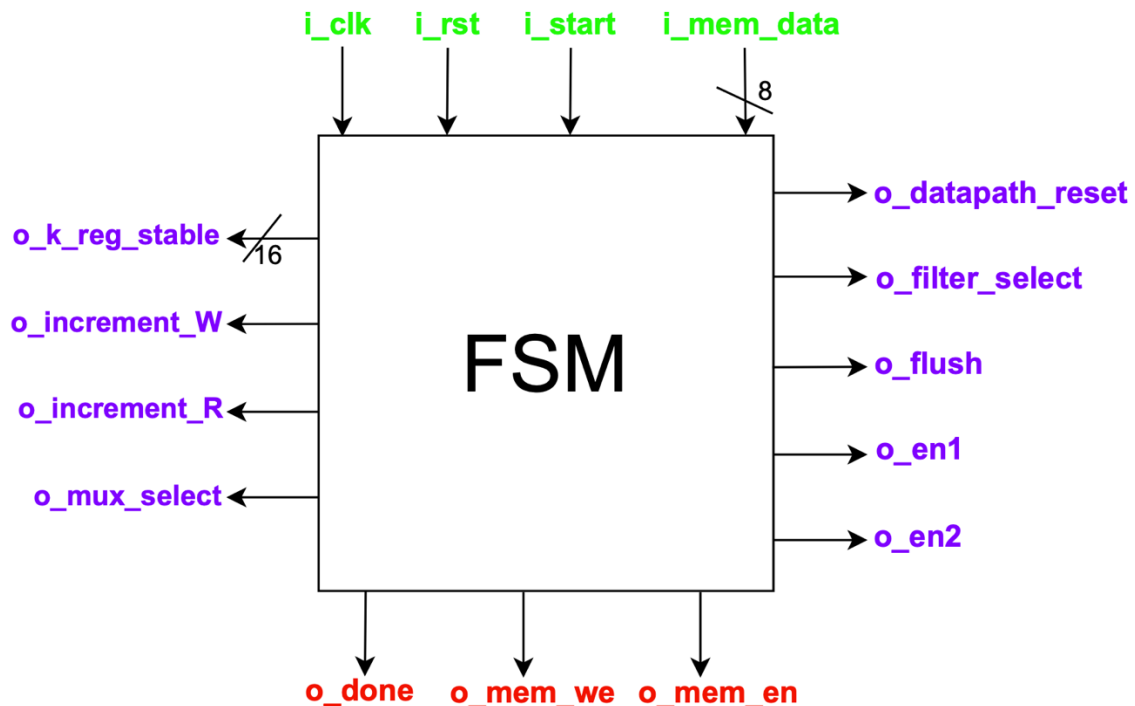


Figure 8: Rappresentazione schematica della macchina a stati.

La **FSM** opera come un controllore centrale che scandisce tutte le operazioni, dalla lettura dei parametri iniziali fino alla scrittura dei risultati finali. Genera una sequenza precisa di segnali di controllo che pilotano i vari blocchi del datapath, a seconda dello stato in cui ci si trova

2.6.1 Interfaccia

- **i_clk**: Segnale di clock per la sincronizzazione.
- **i_rst**: Segnale di reset asincrono.
- **i_start**: Segnale di avvio elaborazione.
- **i_mem_data**: Vettore a 8 bit che riceve il dato letto dalla memoria, usato per catturare parametri (K, S, coefficienti, parole).
- **o_k_reg_stable**: Vettore a 16 bit che espone il valore di K una volta che è stato letto e memorizzato.
- **o_filter_select**: Segnale che comunica a tutti i moduli quale filtro è in uso.
- **o_en1**: Segnale di abilitazione per il **Register_coeff**, per comandare il latch dei coefficienti.
- **o_en2**: Segnale di abilitazione per lo **Shift_register**, per comandare il latch di una nuova parola W.
- **o_increment_R**, **o_increment_W**: Segnali per comandare l'**Address_Manager** ad incrementare rispettivamente il puntatore di scrittura o di lettura.
- **o_mux_select**: Segnale per l'**Address_Manager** che seleziona tra l'indirizzo di lettura (W) e quello di scrittura (R).
- **o_flush**: Segnale per lo **Shift_register** per avviare lo svuotamento della pipeline con zeri.
- **o_done**: Segnale che indica il completamento dell'intera elaborazione.

- **o_datapath_reset**: Segnale di "soft reset" generato all'inizio di ogni elaborazione per reinizializzare lo stato dei componenti del datapath
- **o_mem_en**: Segnale di abilitazione della memoria.
- **o_mem_we**: Segnale di abilitazione lettura/scrittura ('0' lettura, '1' scrittura).

2.6.2 Architettura

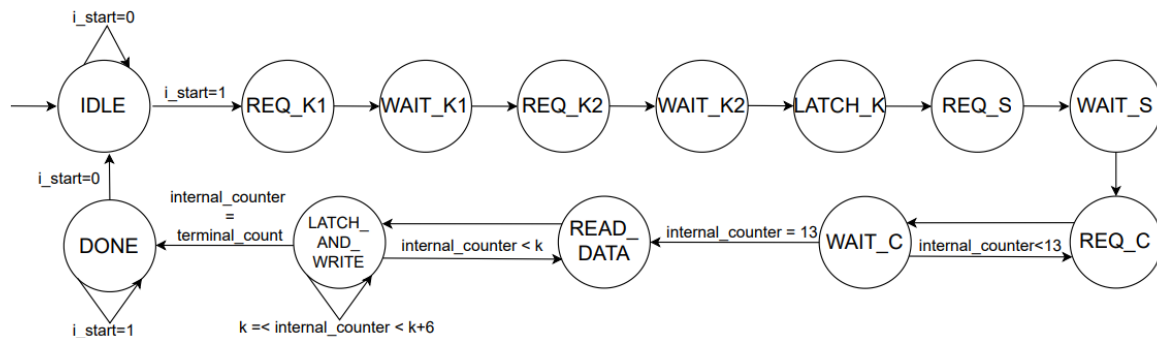


Figure 9: Rappresentazione degli stati e transizioni della macchina a stati.

L'architettura è implementata utilizzando un modello a due processi:

- uno per la gestione delle transizioni di stato
- uno, combinatorio, per la generazione delle uscite.

POLITECNICO
MILANO 1863

2.6.2.1 Processo per le transizioni di stati

```
process(i_clk, i_rst)
  variable terminal_count : natural;
begin
  if i_rst = '1' then
    curr_state <= IDLE;
    internal_counter <= 0;
    k_reg <= 0;
    k1_byte_reg <= (others => '0');
    filter_selected <= '0';
  elsif rising_edge(i_clk) then
    case curr_state is
      when IDLE =>
        if i_start = '1' then
          curr_state <= REQ_K1;
          internal_counter <= 0;
          k_reg <= 0;
          k1_byte_reg <= (others => '0');
          filter_selected <= '0';
        end if;
      when REQ_K1 => curr_state <= WAIT_K1;
      when WAIT_K1 =>
        k1_byte_reg <= unsigned(i_mem_data);
        curr_state <= REQ_K2;
      when REQ_K2 => curr_state <= WAIT_K2;
      when WAIT_K2 => curr_state <= LATCH_K;
      when LATCH_K =>
        k_reg <= to_integer(k1_byte_reg & unsigned(i_mem_data));
        curr_state <= REQ_S;
      [...]
    end case;
  end if;
end process;
```

[versione completa nel file .vhd]

Questo processo, sensibile al clock e al reset asincrono, determina lo stato successivo del sistema (**curr_state**).

- **Avvio e Reset:** Un reset (**i_rst**='1') riporta forzatamente la macchina nello stato IDLE. All'inizio di una nuova elaborazione (**i_start**='1'), la **FSM** esegue un reset completo di tutti i suoi registri interni (**k_reg**, **internal_counter**, etc.) per garantire una partenza pulita, fondamentale per le elaborazioni consecutive.
- **Lettura Preambolo:** La macchina attraversa una sequenza lineare di stati (REQ_K1 → WAIT_K1 → ... → WAIT_S) per leggere i parametri K e S. Successivamente, un ciclo tra gli stati REQ_C e WAIT_C, governato da un contatore interno, legge i 14 coefficienti.
- **Elaborazione e Svuotamento:** La fase di calcolo è un loop tra gli stati READ_DATA e LATCH_AND_WRITE. Questo ciclo continua finché il contatore **internal_counter** non raggiunge un valore **terminal_count** calcolato come **k_reg + 6**. Il valore +6 corrisponde alla profondità della pipeline (latenza di 6 cicli) e assicura che la macchina continui a funzionare

anche dopo aver letto l'ultimo dato W, per poter svuotare la pipeline e scrivere gli ultimi risultati.

- **Fine:** Raggiunto il **terminal_count**, la macchina transita nello stato DONE, da cui tornerà a IDLE solo quando il segnale **i_start** verrà abbassato.

2.6.2.2 Processo per la gestione dei segnali

```
process(curr_state, internal_counter, k_reg, filter_selected)
    variable is_stutter_cycle : boolean;
begin
    o_mem_we <= '0'; o_mem_en <= '0'; o_done <= '0';
    o_en1 <= '0'; o_en2 <= '0';
    o_increment_R <= '0'; o_increment_W <= '0';
    o_mux_select <= '0'; o_flush <= '0'; o_datapath_reset <= '0';
    o_filter_select <= filter_selected;
    o_k_reg_stable <= std_logic_vector(to_unsigned(k_reg, 16));

    case curr_state is
        when IDLE =>
            if i_start = '1' then
                o_datapath_reset <= '1';
            end if;
        when REQ_K1 | REQ_K2 | REQ_S | REQ_C =>
            o_mem_en <= '1';
            o_increment_W <= '1';
        when WAIT_C =>
            o_en1 <= '1';
        when READ_DATA =>
            o_mem_en <= '1';
            o_increment_W <= '1';
        when LATCH_AND_WRITE =>
            if internal_counter <= k_reg then
                o_en2 <= '1';

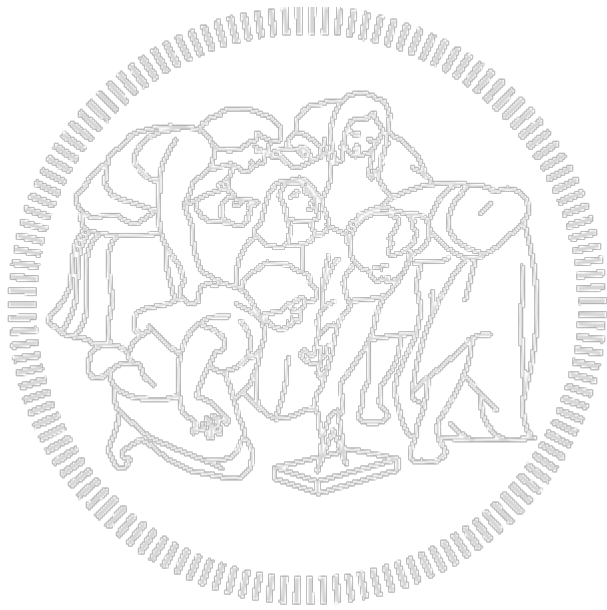
                [...]

                [versione completa nel file .vhd]
```

Questo secondo processo, di natura puramente combinatoria, genera tutti i segnali di controllo in uscita. Per prevenire la creazione di latch, assegna un valore di default inattivo ('0') a tutte le uscite all'inizio del processo.

- **Reset del Datapath:** Nello stato IDLE, se **i_start** è alto, viene generato un impulso di un ciclo di clock su **o_datapath_reset** per reinizializzare **Register_coeff** e **Shift_register**.

- **Stati di Lettura:** Vengono asseriti **o_mem_en** e **o_increment_W** per leggere dalla memoria.
- **Stati di Abilitazione Latch:** Vengono attivati i segnali **o_en1** e **o_en2** per abilitare rispettivamente il caricamento dei coefficienti e delle parole.
- **Logica di Scrittura e Svuotamento:** Nello stato LATCH_AND_WRITE, la logica di scrittura in memoria (**o_mem_we**, **o_mux_select**, **o_increment_R**) viene attivata solo quando **internal_counter** è maggiore o uguale a 6. Questa attesa di 6 cicli è cruciale: dà tempo alla pipeline di calcolo di "riempirsi" con i primi dati validi, risolvendo il problema della "partenza a freddo" e garantendo che solo i risultati corretti vengano scritti. Dopo aver letto l'ultimo dato W (quando **internal_counter** supera **k_reg**), viene attivato il segnale **o_flush** per svuotare la pipeline.



POLITECNICO
MILANO 1863

3 Risultati Sperimentali

3.1 Sintesi

La sintesi del componente è stata effettuata sulla board Artix-7 FPGA xc7a200tfbg484-1 e viene completata senza errori, indicando che il codice è sintetizzabile.

Utilizzando i report generati da Vivado, si è potuta stimare l'area occupata dal componente, il quale richiede un utilizzo di risorse pari a 818 Look-Up Tables (LUTs) e 238 Flip-Flops (FF). Inoltre, la sintesi non ha inferito alcun latch, a conferma della scrittura di un design puramente sincrono. Utilizzando il comando **report_timing_summary** dopo la fase di implementazione, sono stati verificati i vincoli temporali.

Slack (MET) : 5.749 ns (required time - arrival time)

3.2 Simulazioni

Il componente è in grado di eseguire correttamente tutti i test proposti, sia eseguendo la simulazione behavioral, sia quella post-sintesi funzionale.

3.2.1 Testbench fornito dal docente

Il testbench fornito dal docente controlla che una sequenza generica di 24 valori ($K=24$) con filtro di ordine 3 ($S=0$) sia elaborata correttamente e che il componente risponda ai segnali start/done. Il test è stato superato con successo, confermando la corretta implementazione della logica di base del filtro, della gestione della pipeline e degli indirizzi di memoria.

3.2.2 Test di saturazione

È stato creato un test specifico per verificare il meccanismo di saturazione, utilizzando una sequenza di input e coefficienti studiati per generare risultati intermedi al di fuori dell'intervallo $[-128, +127]$. Le forme d'onda sono state analizzate impostando il radix in formato decimale con segno (signed decimal).

Nella seguente immagine, si osserva come un risultato intermedio calcolato dal **Filter**, pari a **+11114**, venga correttamente saturato al valore massimo consentito di **+127**.

Name	Value	3,770,000 ps	3,770,001 ps	3,770,002 ps	3,770,003 ps	3,770,004 ps	3,770,005 ps
tb_start	1						
tb_done	0						
tb_add[15:0]	3529			3529			
tb_o_mem_addr[15:0]	3544			3544			
exc_o_mem_addr[15:0]	3544			3544			
init_o_mem_addr[15:0]	35a7			35a7			
tb_o_mem_data[7:0]	7f			7f			
exc_o_mem_data[7:0]	7f			7f			
init_o_mem_data[7:0]	b2			b2			
tb_i_mem_data[7:0]	71			71			
tb_o_mem_we	0						
tb_o_mem_en	1						
exc_o_mem_we	0						
exc_o_mem_en	1						
init_o_mem_we	1						
init_o_mem_en	1						
scenario_config[0:16]	0,110,0,0,58,76,87,98,-46,0,-52,93,-7,9,40,95,1	0,110,0,0,58,76,87,98,-46,0,-52,93,-7,9,40,95,105					
scenario_input[0:109]	-47,95,1,-21,110,-50,76,-19,101,113,-45,-69,44,-33,102,39,-98,46,-67,118,117,41,-	-47,95,1,-21,110,-50,76,-19,101,113,-45,-69,44,-33,102,39,-98,46,-67,118,117,41,-					
scenario_output[0:109]	127,127,-128,127,-28,127,127,127,127,127,127,-128,127,127,-128,127,-128,-128	127,127,-128,127,-28,127,127,127,127,127,127,-128,127,127,-128,127,-128,-128					
memory_control	1						
CLOCK_PERIOD	20000 ps			20000 ps			
SCENARIO_LENGTH	110			110			
SCENARIO_LEN...H_STL[15:0]	006e			006e			
SCENARIO_ADDRESS	13609			13609			
s_words_window[0:6][7:0]	-21,110,-50,76,-19,101,113	-21,110,-50,76,-19,101,113					
s_coeffs[0:6][7:0]	0,58,76,87,98,-46,0	0,58,76,87,98,-46,0					
s_filter_out_temp[19:0]	11114			11114			
o_mem_data[7:0]	127			127			

Analogamente, nella seguente immagine un risultato intermedio negativo di **-11952** viene saturato al valore minimo consentito di **-128**.

Name	Value	4,050,000 ps	4,050,001 ps	4,050,002 ps	4,050,003 ps	4,050,004 ps	4,050,005 ps
tb_start	1						
tb_done	0						
tb_add[15:0]	3529			3529			
tb_o_mem_addr[15:0]	354b			354b			
exc_o_mem_addr[15:0]	354b			354b			
init_o_mem_addr[15:0]	35a7			35a7			
tb_o_mem_data[7:0]	80			80			
exc_o_mem_data[7:0]	80			80			
init_o_mem_data[7:0]	b2			b2			
tb_i_mem_data[7:0]	9e			9e			
tb_o_mem_we	0						
tb_o_mem_en	1						
exc_o_mem_we	0						
exc_o_mem_en	1						
init_o_mem_we	1						
init_o_mem_en	1						
scenario_config[0:16]	0,110,0,0,58,76,87,98,-46,0,-52,93,-7,9,40,95,1	0,110,0,0,58,76,87,98,-46,0,-52,93,-7,9,40,95,105					
scenario_input[0:109]	-47,95,1,-21,110,-50,76,-19,101,113,-45,-69,44,-33,102,39,-98,46,-67,118,117,41,-	-47,95,1,-21,110,-50,76,-19,101,113,-45,-69,44,-33,102,39,-98,46,-67,118,117,41,-					
scenario_output[0:109]	127,127,-128,127,-28,127,127,127,127,127,127,-128,127,127,-128,127,-128,-128	127,127,-128,127,-28,127,127,127,127,127,127,-128,127,127,-128,127,-128,-128					
memory_control	1						
CLOCK_PERIOD	20000 ps			20000 ps			
SCENARIO_LENGTH	110			110			
SCENARIO_LEN...H_STL[15:0]	006e			006e			
SCENARIO_ADDRESS	13609			13609			
s_words_window[0:6][7:0]	-45,-69,44,-33,102,39,-98	-45,-69,44,-33,102,39,-98					
s_coeffs[0:6][7:0]	0,58,76,87,98,-46,0	0,58,76,87,98,-46,0					
s_filter_out_temp[19:0]	-11952			-11952			
o_mem_data[7:0]	-128			-128			

3.2.3 Test con filtro di ordine 5

Per validare la corretta implementazione di entrambi i filtri, è stato eseguito un test specifico impostando il selettore S a '1' (**s_filter_select**) per utilizzare il filtro di ordine 5. Il componente ha gestito correttamente una lunga sequenza di 349 parole, caricando il set di coefficienti corretto e producendo la sequenza di risultati attesa.

Name	Value	8,950,000 ps	8,950,001 ps	8,950,002 ps	8,950,003 ps	8,950,004 ps	8,950,005 ps
tb_start	1						
tb_done	0						
tb_add[15:0]	76a7			76a7			
tb_o_mem_addr[15:0]	76cc			76cc			
exc_o_mem_addr[15:0]	76cc			76cc			
init_o_mem_addr[15:0]	7814			7814			
tb_o_mem_data[7:0]	2b			2b			
exc_o_mem_data[7:0]	2b			2b			
init_o_mem_data[7:0]	ee			ee			
tb_i_mem_data[7:0]	35			35			
tb_o_mem_we	0						
tb_o_mem_en	1						
exc_o_mem_we	0						
exc_o_mem_en	1						
init_o_mem_we	1						
init_o_mem_en	1						
scenario_config[0:16]	1,93,1,0,120,-117,30,106,-91,0,16,-81,-76,-72,-93,32,-56						
scenario_input[0:348]	-111,-99,78,-54,-79,0,-64,-37,-80,-116,-101,-48	-111,-99,78,-54,-79,0,-64,-37,-80,-116,-101,-48,29,-112,116,-49,54,58,-28,53,73,					
scenario_output[0:348]	127,127,127,127,30,127,127,127,127,127,127,127,127,107,127,-120,1,43,-128,-37,-128,-96,-	127,127,127,127,30,127,127,127,127,127,127,127,127,107,127,-120,1,43,-128,-37,-128,-96,-					
memory_control	1						
CLOCK_PERIOD	20000 ps			20000 ps			
SCENARIO_LENGTH	349			349			
SCENARIO_LEN_H_STL[15:0]	015d			015d			
SCENARIO_ADDRESS	30375			30375			
s_coeffs[0:6][7:0]	16,-81,-76,-72,-93,32,-56			16,-81,-76,-72,-93,32,-56			
s_words_window[0:6][7:0]	-112,116,-49,54,58,-28,53			-112,116,-49,54,58,-28,53			
s_filter_select	1						
o_mem_data[7:0]	43			43			

3.2.4 Test di robustezza

Per testare la robustezza del design, è stata eseguita una simulazione con una sequenza di input di 794 parole. Il componente ha completato l'intera elaborazione senza errori, dimostrando che i contatori interni e la macchina a stati sono in grado di gestire correttamente un numero elevato di cicli, come richiesto per un'applicazione generica.

3.2.5 Elaborazioni consecutive senza reset

Il componente deve essere in grado di eseguire correttamente più elaborazioni in sequenza senza che venga fornito un segnale di reset hardware tra di esse. Questo test è stato progettato per verificare tale capacità. La simulazione avvia una prima elaborazione completa; non appena questa termina, il testbench avvia immediatamente una seconda elaborazione con parametri e indirizzi di memoria diversi. Il superamento di questo test conferma il corretto funzionamento della logica di "soft reset" interna.

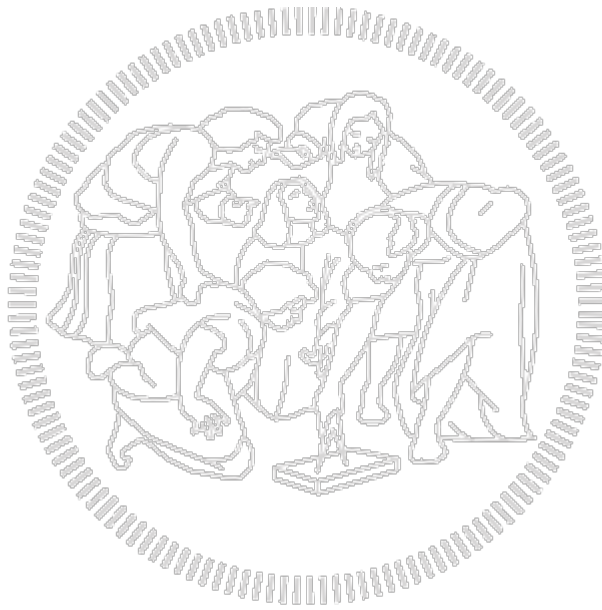
3.2.6 Reset durante l'elaborazione

Lo scopo di questo test è verificare la robustezza di questo meccanismo: durante l'esecuzione di una prima elaborazione, prima che questa giunga al termine, il testbench forza il segnale di reset a '1'

per un certo numero di cicli di clock, interrompendo bruscamente l'operazione. Successivamente, avvia una seconda e diversa elaborazione. Il test è superato in quanto il componente interrompe la prima elaborazione senza scrivere in memoria risultati parziali o corrotti, per poi completare la seconda elaborazione in modo corretto.

3.2.7 Test con lunghezza minima

La lunghezza minima della sequenza di dati W è di 7 byte e questo test mira a verificare il comportamento del componente in questa condizione al contorno. È stato creato uno scenario in cui il valore di K , scritto nel preambolo in memoria, è impostato a 7. Il test verifica che il componente esegua correttamente l'intero ciclo di elaborazione: lettura del preambolo, riempimento della pipeline del datapath, calcolo dei risultati, svuotamento della pipeline e asserzione finale del segnale di fine.



POLITECNICO
MILANO 1863

4 Conclusioni

La prova finale ha portato allo sviluppo di un componente VHDL che implementa con successo la logica di filtraggio differenziale richiesta. L'architettura scelta, basata sulla netta separazione tra un'unità di controllo (FSM) e un datapath dedicato, si è dimostrata robusta e modulare. La verifica del design è stata approfondita tramite una suite di test mirati, che hanno confermato il corretto funzionamento in scenari critici come la saturazione dei dati, le elaborazioni consecutive senza reset e l'interruzione asincrona. Le analisi post-sintesi e post-implementazione hanno inoltre validato la correttezza del design a livello hardware, confermando l'assenza di latch e il rispetto dei vincoli temporali sulla scheda FPGA target.

